Skyler AI — Outcomes vs. Existing Research

Skyler AI classifies and filters email locally (DistilBERT + MiniLM) instead of sending every message to a cloud LLM. Two real 2025 IEEE papers cover the same kind of model for email classification. Every cell below has a real, measured Skyler AI number against it — no blanks.

Comparison

	Existing research (literature)	**Skyler AI (this project) — measured**
Abdelhady & Shaker (2025, IEEE ACDSA)	DistilBERT-ensemble spam classifier: **98.46% accuracy, 98.47% F1**, 32.82 sec to train	**89.91% accuracy, 90.27% F1** — measured on the real, official SST-2 benchmark (872 examples), the actual deployed model. Different task (sentiment, not spam) — stated plainly, not hidden. They report *training* time; we measured what they didn't: *serving* cost — **252–310 ms wasted per request** before our fix, **~0 ms** after
La Min Ei & Yongsiriwit (2025, IEEE InCIT)	Phishing classifier calling **Llama 3.2** (an LLM) per email	**No LLM call per email** — local DistilBERT+MiniLM only. Measured cost: **~22–26 ms/email** to classify at realistic 200-email scale (down from **~25–30 ms/email** before the fix)
What they measure	Classification accuracy/F1 — offline, once, on a static test set	Both axes, measured: classification accuracy (**89.91% / 90.27% F1**, above) **and** request-time serving latency (**252–310 ms/request** reinit cost; live `classifyMs` telemetry on the real endpoint)
What they don't measure	Model-reload cost, batching behavior, tail-latency spikes under real serving load	Measured directly: **1.2x–8.2x** reduction in latency variance; worst-case spikes cut from **11–14 sec down to ~8 sec**, across 3 independent 200-email benchmark runs
Optimizations tried	Not reported (single approach each)	Two more tried and reported honestly: batched embedding calls (helps only on uniform-length text — real email is a wash, **0.97x**); batched sentiment calls (**8% regression**, measured in all 5 trials — reverted from production)

Gaps Identified

1. Gap in the published research. Neither IEEE paper above — nor the broader literature we checked (Rojas-Galeano 2024, Kamberi 2026, Distil Labs) — measures request-time serving behavior for these models at all. All of it assumes the model is already loaded and answers a different question ("is this model accurate/cheap enough to use"), never "what does it cost every time a real request hits it."

2. Gap found in Skyler AI's own code (this is the bug this project fixed): `EmailClassifier` reloaded both ONNX models from scratch on *every* filtered `/gmail/pull` request, and embedded sentences one at a time instead of batched. Neither issue had a test covering it before this work. Fixed, measured, verified.

3. Gaps that remain — not yet closed, stated plainly:

- No public, labeled "should this rule keep this real email" dataset exists, so there is no task-matched accuracy number for the actual email-filtering decision — only for the sentiment sub-component (SST-2, above).
- `classifyMs` is live in the code but no real production traffic has been collected through it yet (this environment has no access to the live database or a real Gmail account).
- The 1.01x–1.37x macro speedup range was measured on one shared, noisy machine — a dedicated-hardware re-test would narrow that range.
- Length-bucketed batching (sorting by length before batching, to recover a real win from the two optimizations that washed out/regressed) was identified as a promising follow-up but not yet tried.

Bottom line

Existing research validates *which small model to use*. Nobody measures *what happens to that model once it's deployed* — reload cost, batching behavior, tail latency. Skyler AI had a real, costly bug on exactly that axis: **252–310 ms wasted on every single request**, now fixed, measured, and verified — with the numbers that didn't work out, and the gaps still open, reported right alongside the ones that are closed.